Telephone Broadcasting Beyond Trees: Polynomial and Quasi-Polynomial Approximation Schemes

Anonymous author
Anonymous affiliation

Abstract

Distributing information throughout a network is a basic problem in communication systems. In Telephone Broadcasting, one source vertex initially holds a message, and communication proceeds in rounds, with each informed vertex allowed to transmit the message to at most one uninformed neighbor per round. The objective is to inform all vertices in the minimum number of rounds. While an optimal broadcast schedule can be computed in polynomial time on trees, the problem becomes NP-hard even on cactus graphs and graphs of treewidth two. This motivates the study of approximation schemes on graph classes that retain a tree structure. In the concluding remarks of their ICALP 2025 paper [1], Aminian et al. explicitly asked whether Telephone Broadcasting admits a polynomial-time approximation scheme on cactus graphs and graphs of bounded pathwidth. We resolve the question for cactus graphs and make progress on the bounded-pathwidth question by establishing a quasi-polynomial-time approximation scheme for the more general class of graphs of bounded treewidth.

As the first result, we give a randomized polynomial-time approximation scheme for Telephone Broadcasting on cactus graphs. For every fixed $\epsilon \in (0, 1/2]$, our algorithm constructs a schedule of length at most $(1+\epsilon)$ times the optimum with probability at least $(1-n^{-2})$, in time $n^{\mathcal{O}(\epsilon^{-2}\log(1/\epsilon))}$. We then obtain a deterministic quasi-polynomial-time approximation scheme for graphs of bounded treewidth. Given a tree decomposition of width w, the algorithm achieves a $(1+\epsilon)$-approximation in time $n^{\mathcal{O}((w+1)\log n/\epsilon)}$. These results provide approximation guarantees as close to optimal as desired on these graph classes, even though finding an optimal broadcast schedule is NP-hard.

2012 ACM Subject Classification Theory of computation → Approximation algorithms analysis; Theory of computation → Graph algorithms analysis; Theory of computation → Dynamic programming; Theory of computation → Rounding techniques

Keywords and phrases Telephone Broadcasting, Approximation Algorithms, Randomized Algorithms, Cactus Graphs, Treewidth, PTAS, QPTAS

1 Introduction

Broadcasting is a basic information dissemination problem in communication networks. A message initially held by one vertex must reach the entire network, while each informed vertex has limited capacity to transmit it. In the telephone model, the network is a connected, undirected graph $G = (V, E)$ with a designated source $s \in V$. In each round, every vertex informed at the beginning of the round may send the message to at most one uninformed neighbor. The goal of Telephone Broadcasting is to minimize the number of rounds needed to inform all vertices. We denote this minimum by $b(G, s)$ and write $n = |V|$. Despite its simple formulation, the problem captures the interaction between network topology and limited communication capacity of individual vertices, and has been studied extensively in communication networks and distributed computing.

Telephone Broadcasting

Input: A connected graph G, a source vertex $s \in V(G)$, and an integer t.
Question: Decide whether the message can reach every vertex within t rounds, that is, whether $b(G, s) \leq t$.

Telephone Broadcasting was studied by Slater et al. [9], who showed that the problem is NP-complete on general graphs but can be solved in polynomial time on trees. The problem is computationally difficult on general graphs. They established its NP-completeness, and Elkin and Kortsarz [6] showed that, for every fixed $\delta > 0$, obtaining a $(3 - \delta)$-approximation is NP-hard. The best known polynomial-time approximation guarantee is $\mathcal{O}(\log n / \log \log n)$, due to Elkin and Kortsarz [7, 1]; whether a constant-factor approximation exists remains an important open problem. These results motivate the study of graph classes whose structure can support stronger approximation guarantees. Trees provide a natural starting point. Unlike general graphs, Telephone Broadcasting is polynomial-time solvable on trees [9]. Rooting a tree at the source fixes the vertex from which each non-source vertex receives the message. The remaining task is to order the calls made by each vertex. If the child subtrees require $q_1 \geq q_2 \geq \cdots \geq q_d$ additional rounds after their roots are informed, an optimal order calls them in this order, giving a completion time of $\max_{i \in [d]}(i + q_i)$. Thus, the problem can be solved recursively by combining the completion times of the child subtrees. A natural question is how far this tractability extends when the input graph retains a tree structure but admits additional routing choices.

We study two such generalizations: cactus graphs and graphs of bounded treewidth. A cactus is a connected graph in which every block is either a bridge or a cycle. Its blocks are arranged in a tree structure, although a cycle can be informed along two different directions. Graphs of bounded treewidth generalize trees by admitting tree decompositions with small bags. Every tree has treewidth at most one, and every cactus has treewidth at most two. Both classes therefore preserve a form of tree structure. However, broadcasting requires choosing a spanning tree and coordinating the calls across its branches, and these choices are absent when the input itself is a tree. Recent hardness results show that this distinction is substantial. Tale [10] proved that Telephone Broadcasting is NP-complete even on graphs with feedback vertex number one, and hence on graphs of treewidth two. Aminian et al. [1] established NP-completeness on cactus graphs of pathwidth two. Egami et al. [5] further showed hardness on cacti that become a disjoint union of paths after deleting one vertex. Consequently, even very simple extensions of trees do not admit polynomial-time exact algorithms unless P = NP. The relevant question is whether the tree structure can instead be used to obtain schedules arbitrarily close to optimal.

There has been substantial progress toward this question. Polynomial-time exact algorithms are known for unicyclic graphs and certain restricted families of cactus graphs [4]. For arbitrary cactus graphs, Aminian et al. [1] gave a polynomial-time 2-approximation algorithm. They also established an $\mathcal{O}(4^p)$-approximation guarantee for graphs of bounded pathwidth p, which Egami et al. [5] subsequently improved to $\mathcal{O}(p)$. These guarantees are constant for every fixed p. In the concluding remarks of Aminian et al.s ICALP 2025 paper [1], the authors explicitly asked whether polynomial-time approximation schemes exist for cactus graphs and graphs of bounded pathwidth. Here, the graph classes are cactus graphs and graphs of bounded pathwidth. More recently, Bringolf et al. [3] gave efficient PTASs (EPTASs) for cycle-star graphs (also called k-cycle graphs), consisting of cycles sharing a single vertex, and for melon graphs (also called k-path graphs), consisting of paths with common endpoints. Their result for cycle-star graphs addresses a restricted family of cacti. Extending such a guarantee to arbitrary cacti requires handling cycles arranged recursively throughout the graph.

1.1 Related Work

Telephone Broadcasting has been studied through exact algorithms, approximation algorithms, and parameterized complexity. Fomin, Fraigniaud, and Golovach [8] initiated a systematic study of its structural parameterizations. They gave a $3^n n^{O(1)}$-time exact algorithm and proved fixed-parameter tractability for feedback edge set number, vertex cover number, and $n - t$, where t is the broadcast deadline. However, other natural structural parameters do not yield the same tractability. Tale [10] proved NP-completeness on graphs with feedback vertex set number one, and hence on graphs of treewidth two. Egami et al. [5] established NP-completeness on cactus graphs that become path forests after deleting one vertex, and on graphs of bounded tree-depth. They also obtained fixed-parameter algorithms for vertex integrity and distance to a clique, together with parameterized approximation algorithms for clique cover number and cluster vertex deletion number. Particularly relevant to our work is their $2^{O(wt)} n^{O(1)}$-time exact algorithm [5], where w is the treewidth and t is the broadcast deadline.

1.2 Our Contributions

We give approximation schemes for Telephone Broadcasting on both cactus graphs and graphs of bounded treewidth.

A Randomized PTAS on Cactus Graphs. Our main result is a randomized PTAS for arbitrary cactus graphs. For every fixed $\varepsilon \in (0, 1/2]$, the algorithm constructs a broadcast schedule of length at most $\lfloor (1+\varepsilon) b(G, s) \rfloor$ with probability at least $1 - n^{-2}$, in time $n^{\mathcal{O}(\varepsilon^{-2} \log(1/\varepsilon))}$. This answers affirmatively, in the randomized setting, the cactus-graph part of the question posed by Aminian et al. [1] In particular, the guarantee applies without restrictions on the number of cycles, their lengths, or the number of cycles incident with a vertex.

▶ **Theorem 1.1.** *Fir every fixed $\epsilon \in (0, 1/2]$, there is a randomized algorithm for* Telephone Broadcasting *problem on a n vertex cactus graph that, with probability at least $1 - n^{-2}$, outputs a broadcast schedule of length at most $\lfloor (1+\epsilon) OPT \rfloor$ with runtime $n^{\mathcal{O}(\epsilon^{-2} log(1/\epsilon))}$.*

Hence Telephone broadcasting admits a randomized PTAS on a cactus graph.

A QPTAS on Graphs of Bounded Treewidth. Our second result is a deterministic quasi-polynomial-time approximation scheme for graphs of bounded treewidth. Given a

tree decomposition of width w and $\varepsilon \in (0, 1]$, the algorithm constructs a schedule of length at most $(1 + \varepsilon)b(G, s)$ in time $n^{\mathcal{O}((w+1)\log n/\varepsilon)}$. For every fixed w and ε, this running time is quasi-polynomial. The result also applies to graphs of bounded pathwidth, providing a QPTAS for the second class in the ICALP question. Together, the two results show that the tree structure supports arbitrarily accurate approximation even though exact broadcasting becomes NP-hard already at treewidth two [1].

▶ **Theorem 1.2.** *Given a connected n-vertex graph G, a source vertex $s \in V(G)$, a tree decomposition of G of width tw, and $\epsilon \in (0, 1]$, there exists a deterministic algorithm that constructs a broadcast schedule of length at most $(1 + \epsilon)b(G, s)$ in time $n^{\mathcal{O}((w+1)\log n/\epsilon)}$. Consequently,* Telephone Broadcasting *admits a QPTAS on graphs of bounded treewidth.*

1.3 Overview of the Techniques

For cactus graphs, we exploit the block-incidence tree directly. Every broadcast tree omits exactly one edge from each cycle. Choosing this edge creates one or two branches that must be activated from the parent articulation vertex. We express these activations as unit-time jobs whose tails represent the time needed to complete their branches. The local problem is to select one option for each child cycle and order the resulting jobs. We handle a bounded number of choices with early deadlines by enumeration and formulate the remaining choices as a linear program with prefix constraints on the number of jobs due by each deadline. Randomized rounding selects one option per cycle, while concentration bounds control the load on every relevant prefix. We adjust the deadlines to account for the approximate time needed to finish each branch, so the error does not keep growing as we move up the block-incidence tree. This gives a polynomial-time algorithm for fixed (ε), with an overall approximation factor of $(1 + \varepsilon)$.

For graphs of bounded treewidth, we organize the computation around a tree decomposition and control the approximation error through local call ranks. We start from the dynamic programming approach of Egami et al. [5], whose exact algorithm runs in $2^{O(wt)}n^{O(1)}$ time for a target broadcast time t. Explicitly recording the occupied call slots of every bag vertex makes the state space exponential in t. We instead partition the possible local call ranks into $O(\log n/\varepsilon)$ geometrically growing buckets. For each bag vertex, the dynamic program records its activation time, whether its parent has been selected, and the number of outgoing calls assigned to each bucket. Rounding a call rank to the upper endpoint of its bucket increases it by a factor of at most $1 + \varepsilon$. A vertex receives the message after the delays along its path from the source have added up. Rounding increases each delay by at most $(1 + \varepsilon)$, so their sum increases by at most the same factor. Counting calls in each bucket helps schedule each vertex calls in different rounds while keeping the state space quasi-polynomial.

Organization of the Paper. In Section 2, we introduce the notation and preliminary concepts used throughout the paper. In Section 3, we present our randomized PTAS for Telephone Broadcasting on cactus graphs, including the main algorithm, its correctness, and running-time analysis. In Section 4, we develop a deterministic QPTAS for graphs of bounded treewidth using a bucketing-based dynamic programming approach and establish its approximation guarantee and running time. Finally, in Section 5, we conclude the paper and discuss possible directions for future research.

2 Preliminaries

Throughout the paper, we consider a connected, undirected graph $G = (V, E)$ with a designated source vertex $s \in V$, and let $n = |V|$. For a positive integer k, we write $[k] = \{1, 2, \ldots, k\}$. We use $\mathcal{O}(\cdot)$ to denote asymptotic upper bounds up to constant factors.

Telephone Broadcasting. In the Telephone Broadcasting problem, initially only the source vertex s has the message. Communication proceeds in synchronous rounds, where every vertex informed at the beginning of a round can inform at most one uninformed neighbor. A broadcast schedule specifies the calls made in each round, and its completion time is the number of rounds needed to inform all vertices. We denote the minimum completion time by $b(G, s)$, and write $\mathrm{OPT} = b(G, s)$. For a vertex v, let $a(v)$ denote the round in which v becomes informed, with $a(s) = 0$. Since every non-source vertex is informed for the first time by exactly one call, the edges used for these first transmissions form a spanning tree rooted at s, called a broadcast tree.

Cactus Graphs. A graph is called a cactus if every edge belongs to at most one simple cycle. Equivalently, every block of a connected cactus graph is either a bridge or a cycle. Here, a block is a maximal connected subgraph without a cut vertex, with bridges considered as blocks. For a cactus graph, we use its block-incidence tree T_{tree}, whose nodes correspond to the original vertices and the blocks of G. An original vertex is adjacent to a block node exactly when it belongs to that block. We root T_{tree} at the source s.

Tree Decompositions. A tree decomposition of G is a pair $(\mathcal{T}, \{B_x\}_{x \in V(\mathcal{T})})$, where $\mathcal{T}$ is a tree and each $B_x \subseteq V$ is called a bag, satisfying the following conditions:

1. Every vertex of G belongs to at least one bag.
2. For every edge $uv \in E$, there exists a bag containing both u and v.
3. For every vertex $v \in V$, the nodes whose bags contain v induce a connected subtree of $\mathcal{T}$.

The width of a tree decomposition is $\max_{x \in V(\mathcal{T})} |B_x| - 1$, and the treewidth of G is the minimum width over all its tree decompositions. For our dynamic programming algorithm, we use a nice edge tree decomposition, where the nodes are of five types: leaf, introduce vertex, introduce edge, forget vertex, and join nodes. Each edge of G is introduced exactly once, and the algorithm processes the decomposition in a bottom-up manner.

Approximation Schemes. For $\epsilon > 0$, a $(1+\epsilon)$-approximation algorithm for a minimization problem returns a feasible solution of cost at most $(1 + \epsilon)\mathrm{OPT}$, where OPT denotes the optimal value. A polynomial-time approximation scheme (PTAS) has polynomial running time for every fixed $\epsilon > 0$, whereas a quasi-polynomial-time approximation scheme (QPTAS) has quasi-polynomial running time for every fixed $\epsilon > 0$.

We now describe the algorithms in detail, starting with the randomized PTAS for cactus graphs, followed by the QPTAS for graphs of bounded treewidth.

3 A Randomized PTAS on a Cactus Graph for Broadcasting

In this section, we give a randomized PTAS for the Telephone Broadcasting problem on cactus graphs. The aim is to obtain a $(1 + \epsilon)$-approximate broadcast schedule in polynomial time for every fixed $\epsilon > 0$. Notice that every nontrivial block of a cactus graph is either a bridge or a cycle, and any two cycles share at most one vertex. These are the main structural properties that our algorithm uses.

3.1 Overview of the Algorithm

Our algorithm starts by building a block-incidence tree T_{tree} rooted at the source vertex s. This is a bipartite graph (B, N, E'), where B contains one node for each block of $G = (V, E)$, $N = V$ contains the original vertices, and $(b, v) \in E'$ exactly when v belongs to block b. Rooting this tree at s identifies the vertex through which the message must enter each block and allows us to process the graph from the leaves toward the source. Notice that the edges through which vertices first receive the message in a broadcast schedule form a spanning tree of G. For a cactus graph, every spanning tree contains all bridges and omits exactly one edge from each cycle. Thus, our task has two parts: choosing an edge to delete from each cycle and deciding the order in which each vertex makes its calls. Both choices affect the time needed to complete the broadcast. We handle these choices using bottom-up dynamic programming on T_{tree}. For an original vertex v, each child block is either a bridge or a cycle. A bridge leads to one branch that must be activated by a call from v. Deleting an edge from a child cycle creates one or two branches that must be activated from v. We consider the different deletion choices and the time needed to finish the resulting branches. When combining these branches, we must also account for their starting times, since v can make only one call per round. Our local approximation procedure selects and orders these calls while controlling the increase in the overall broadcast time. The algorithm considers every target value $T \in \{0, 1, \ldots, n-1\}$. If the given instance admits a broadcast schedule of length at most T, it constructs, with high probability, a schedule that completes within $H = (1+\epsilon)T$ rounds. In particular, considering $T = b(G, s)$ gives the desired approximation guarantee. Before describing the dynamic program and the local approximation procedure, we state some basic observations.

3.2 Flowchart of the Algorithm

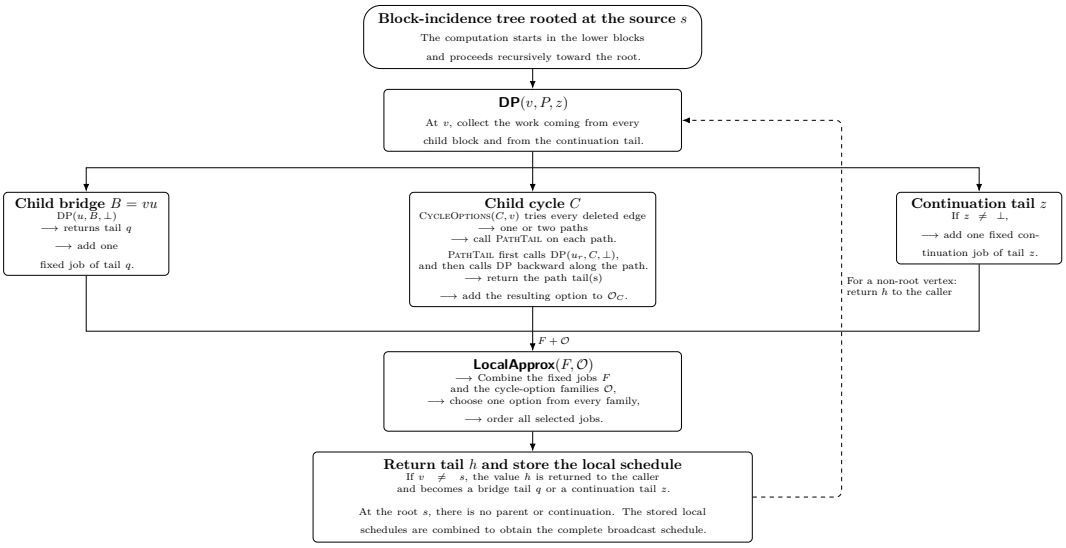


Figure 1 Overall recursive flow of the cactus broadcast algorithm.

3.3 Algorithm Preprocess

Notice that, every vertex $v \neq s$ (other than the source) is first informed by exactly one call and through a particular edge. We keep only those edges by which a non-source vertex

getting informed and orient them away from s.

▶ **Observation 3.1.** *The graph induced by only those edges, which are used to inform a vertex for the first time, in any broadcast schedule, forms a spanning tree.*

So we can always think of a broadcast schedule as first forming a spanning tree rooted at s and then for each vertex, choosing an order in which all its children are informed.

▶ **Observation 3.2** (♣[1])**.** *Let $G(V, E)$ be a connected cactus. A spanning tree of G is obtained by deleting exactly one edge from every cycle and keeping every bridge. Conversely, deleting one edge from every cycle produces a spanning tree.*

Hence, a cycle C of length ℓ in G gives rise to ℓ options, one for each possible edge to delete. Thus, every cycle block of G defines an option family. Now, suppose a vertex v has r children $u_1, u_2, \ldots, u_r$ in a rooted spanning tree. We may assume that v informs its children in consecutive rounds after it becomes informed, since delaying a call cannot improve the completion time. Let x_i be the round in which v informs u_i, measured from the time v becomes informed. Then $x_i \in [r]$, and $x_i \neq x_j$ whenever $i \neq j$. Suppose q_i is the number of additional rounds needed to inform the subtree rooted at u_i after u_i becomes informed. This branch then finishes after $x_i + q_i$ rounds from the time v becomes informed. Our aim is to order the calls from v so that all its child subtrees finish as early as possible. Mathematically, the objective is

$$\min_{\pi} \max_{i \in [r]} \big(i + q_{\pi(i)}\big),$$

where π is a permutation of $[r]$. This leads to the following observation.

▶ **Observation 3.3.** *For a fixed rooted tree, an optimal local order at a vertex is obtained by sorting the children in non-increasing order based on their additional rounds needed to finish the branch i.e. q_i.*

Using these observations, we now introduce the definitions needed to describe our dynamic programming algorithm.

▶ **Definition 3.4** (Block-Incidence Tree.)**.** *Let G be a connected cactus graph with at least two vertices. Every block of G is either a bridge or a cycle. The block-incidence tree T_{tree} has one node for each original vertex and one node for each block of G. An original vertex v is adjacent to a block node B exactly when v belongs to B. This incidence graph is a tree, which we root at the source vertex s.*

▶ **Definition 3.5** (Jobs and Tail)**.** *A job j represents one call from the current vertex v to a child u in the selected broadcast tree. Each job takes one round. Its tail ($q_j \geq 0$) is the number of additional rounds needed by the chosen schedule to finish the branch rooted at u, after u receives the message. Thus, if v makes this call in position x, the branch finishes after $x + q_j$ rounds from the time v becomes informed. Child blocks contribute jobs in the following two ways.*

Type 1: Fixed Jobs. Suppose v has a child bridge block $B = vu$. The message must pass from v to u to reach the subtree below B. Therefore, this block contributes one mandatory job, representing the call from v to u. We call this a fixed job. Its tail is the time needed to finish the subtree rooted at u after u becomes informed.

[1] ♣ Proofs of statements marked with this symbol are deferred to the appendix.

Type 2: Cycle-option Families. Suppose v has a child cycle block C of length ℓ, with edges $e_1, e_2, \ldots, e_\ell$. Each choice of an edge e_i to delete determines an option J_{e_i}. If e_i is incident with v, this option contains one job; otherwise, it contains two jobs. These jobs represent the calls from v that start the branches formed by deleting e_i. Each job's tail accounts for completing its branch, including the subtrees attached to that branch. The cycle therefore contributes the option family $O_C = J_{e_1}, J_{e_2}, \ldots, J_{e_\ell}$. The algorithm must select exactly one option from O_C.

▷ Claim 3.6 (♣). Every option $o_e \in O_C$, corresponding to the deletion of an edge e from a child cycle C of v, creates at most two jobs at v.

By the previous claim, choosing an edge produces either one long path or two paths. The two cases above account for jobs from child blocks. However, if v must inform the next vertex on a path obtained from its parent cycle block, we represent this call by an additional fixed job, called a continuation job.

Type 3: Continuation jobs. Let P be the parent cycle block of a vertex v. After selecting an option $J_e \in O_P$, consider one of the resulting paths $u_0, u_1, \ldots, u_r$, where u_0 is the vertex through which the message enters P. If $v = u_i$ for some $1 \leq i < r$, then v must inform the next vertex u_{i+1} on this path. We represent the call $u_i \to u_{i+1}$ by a fixed job, called a *continuation job* for the parent block P. The last vertex u_r has no continuation job.

For a vertex v, let F denote the set of all fixed jobs at v, including its continuation job, if one exists. Let $O = \{O_C : C \text{ is a child cycle block of } v\}$ be the collection of option families at v. We must select exactly one option $J_e \in O_C$ from each family and order the resulting jobs together with the fixed jobs in F. However, directly enumerating all combinations of options leads to the following difficulty.

Obstruction for PTAS. Suppose a vertex v has m child cycle blocks, each of length at most ℓ. Selecting one edge to delete from each child cycle block gives up to ℓ^m combinations. Thus, directly enumerating all combinations is not polynomial-time in general and does not yield a PTAS.

We address this difficulty using bottom-up dynamic programming on the block-incidence tree. For each original vertex v, we consider states of the form $\mathrm{DP}(v, P, z)$, where P is the parent block of v, and z is the tail of the continuation job, if one exists. The state stores the completion time of the schedule constructed for the subtree rooted at v, together with the continuation branch represented by z, measured from the time v becomes informed. When no continuation job is present, we set $z = \perp$ and use $\mathrm{DP}(v, P, \perp)$.

To compute these states, we use three procedures: LocalApprox, which selects and orders jobs; PathTail, which computes the tail of a path branch; and CycleOptions, which constructs the option family for a child cycle block. We describe these procedures next.

3.4 Core of the Algorithm

We now describe the three main procedures of our algorithm. First, LocalApprox selects and orders the jobs using linear programming and randomized rounding. We then describe PathTail and CycleOptions, which compute the branch tails and construct the option families required by the local procedure.

3.4.1 Local Approximation via Randomized Rounding

Given a set F of fixed jobs, a collection $\mathcal{O}$ of option families, and a tail q_j for every job j, the objective of LocalApprox is to select one option from each family and order the resulting jobs

together with the fixed jobs so as to minimize $\max_j(x_j + q_j)$, where x_j is the position of job j in the ordering. We now describe how to approximate this objective using randomization.

Fix $\epsilon \in (0, 1/2]$, and define $R = 1+\epsilon$ and $\sigma = 1+\epsilon/2$. Each call to LocalApprox considers all target values $T \in \{0, 1, \ldots, n-1\}$. For a fixed T, set $H = \lfloor RT \rfloor$. Let q_j denote the exact tail of job j, and let $\hat{q}_j$ be the approximate tail returned by a lower recursive call. Our aim is to show that, if the local instance can finish within T using the exact tails and $\hat{q}_j \le \lfloor Rq_j \rfloor$ for every job, then the algorithm constructs, with high probability, a schedule that finishes within H. The procedure returns the smallest completion time among the schedules it finds. This value is used as the tail for the current vertex v. For each job j, define $D_j = H - \hat{q}_j$. Thus, D_j is the latest position at which job j can be scheduled while allowing its branch to finish within H. If $D_j < 1$, the job cannot meet this target. We therefore discard any cycle option containing such a job and reject the current target if a fixed job has $D_j < 1$ or an option family becomes empty. For each remaining job, introduce the transformed deadline $d_j = \max\{d \in \mathbb{Z}_{\ge 0} : \lfloor \sigma d \rfloor \le D_j\}$. This transformation leaves room for the later rounding step: even if job j is scheduled as late as round $\lfloor \sigma d_j \rfloor$, it still meets its deadline D_j. For a fixed selection of cycle options, let J contain all selected jobs and fixed jobs, and define $L(t) = |\{j \in J : d_j \le t\}|$. Thus, $L(t)$ counts the jobs whose transformed deadlines are at most t, and its value changes only when t reaches one of these deadlines.

We next handle the jobs with early transformed deadlines. Set $K = \lceil C\epsilon^{-2} \log(1/\epsilon) \rceil$, where C is a sufficiently large constant. To avoid enumerating choices for every child cycle block, as discussed earlier, we guess a set A of at most K child cycle blocks of v. For each cycle block in A, we also guess one edge to delete, thereby selecting one option from its option family. These guesses are intended to cover the cycle options that contribute jobs with transformed deadlines at most K. We then treat all jobs in the selected options as fixed jobs and add them to F.

▶ **Observation 3.7 (♣).** *In any feasible integral schedule, every job j with deadline d_j, at most K elements of $\mathcal{O}$ can have their selected option (edge) that creates a job with deadline (d_j) at most K.*

Therefore, if a feasible integral solution exists, one of our guesses correctly identifies all option families whose selected options contain a job with $d_j \le K$, together with those selected options. For each remaining child cycle block $C \notin A$, we remove every option from O_C that contains a job with $d_j \le K$. We then use a linear program to handle the remaining (LP) option families.

Rounded LP. Fix a guess A together with one selected option for each child cycle block in A. Let F contain all fixed jobs, including those contributed by child bridge blocks, the continuation job if present, and the jobs in the guessed cycle options. For each threshold t, define $F(t) = |\{j \in F : d_j \le t\}|$. For every remaining child cycle block $C \notin A$ and every surviving option $J_e \in O_C$, introduce a variable $x_{C,e}$, representing the fractional selection of J_e. Define $a_{C,e}(t) = |\{j \in J_e : d_j \le t\}|$. Since each cycle option contains at most two jobs, $a_{C,e}(t) \in \{0, 1, 2\}$. In particular, it is zero when all jobs in J_e have transformed deadlines greater than t.

Let $\mathcal{D}$ be the set of distinct transformed deadlines among the fixed jobs and the jobs in the surviving options. We solve the following linear feasibility program:

$$\sum_{J_e \in O_C} x_{C,e} = 1 \qquad \text{for every remaining child cycle block } C \notin A, \tag{1}$$

$$x_{C,e} \ge 0 \qquad \text{for every } C \notin A \text{ and } J_e \in O_C, \tag{2}$$

$$F(t) + \sum_{C \notin A} \sum_{J_e \in O_C} a_{C,e}(t) x_{C,e} \leq t \qquad \text{for every } t \in \mathcal{D}. \tag{3}$$

Constraints (1) and (2) fractionally select one option from each remaining option family. Constraint (3) ensures that, for every relevant threshold t, the total fractional number of jobs with transformed deadlines at most t, including the fixed jobs, does not exceed t.

▶ **Observation 3.8.** *For every threshold $t \leq K$, every remaining child cycle block $C \notin A$, and every surviving option $J_e \in O_C$, we have $a_{C,e}(t) = 0$.*

Given a feasible solution to the LP, the algorithm selects exactly one option $J_e \in O_C$ from each remaining option family, choosing J_e with probability $x_{C,e}$. These choices are made independently across different option families. Let $X_{C,e} \in \{0, 1\}$ be the indicator variable for selecting J_e. Thus, $\Pr[X_{C,e} = 1] = x_{C,e}$, $\sum_{J_e \in O_C} X_{C,e} = 1$. After rounding, the number of fixed and selected jobs with transformed deadlines at most t is $L(t) = F(t) + \sum_{C \notin A} \sum_{J_e \in O_C} a_{C,e}(t) X_{C,e}$.Since the selected options are random, $L(t)$ is a random variable. For each relevant threshold t and each remaining child cycle block $C \notin A$, define $Y_C(t) = \sum_{J_e \in O_C} a_{C,e}(t) X_{C,e}$. Thus, $Y_C(t)$ counts the jobs with transformed deadlines at most t in the option selected from O_C. Since exactly one option is selected and each option contains at most two jobs, $Y_C(t) \in \{0, 1, 2\}$. By linearity of expectation,

$$\mathbb{E}[Y_C(t)] = \sum_{J_e \in \mathcal{O}_C} a_{C,e}(t) \mathbb{E}[X_{C,e}] = \sum_{J_e \in \mathcal{O}_C} a_{C,e}(t) x_{C,e}.$$

Consequently, the LP constraint at threshold t gives

$$\mathbb{E}[L(t)] = F(t) + \sum_{C \notin A} \mathbb{E}[Y_C(t)] = F(t) + \sum_{C \notin A} \sum_{J_e \in \mathcal{O}_C} a_{C,e}(t) x_{C,e} \leq t.$$

The next lemma proves that such a solution exists with non-zero probability.

▶ **Lemma 3.9 (♣).** *For a sufficiently large constant C in the definition of K, one application of randomized rounding to a feasible LP solution satisfies $L(t) \leq \lfloor \sigma t \rfloor$ simultaneously for all relevant thresholds t, with non-zero probability.*

The preceding lemma guarantees that randomized rounding can satisfy the required prefix bounds. We next show that these bounds ensure that every selected job meets its deadline.

▷ Claim 3.10 (♣). Suppose that the selected jobs are ordered by nondecreasing transformed deadlines d_j, and $L(t) \leq \lfloor \sigma t \rfloor$ for every relevant threshold t. Then every selected job j satisfies $x_j \leq D_j$. Consequently, $x_j + \hat{q}_j \leq H$.

The preceding claim shows that ordering the selected jobs by their transformed deadlines preserves the required completion-time bound. We now combine this with the rounding guarantee to establish the correctness of LocalApprox.

▶ **Lemma 3.11 (♣).** *Consider a local instance at some vertex v. Let q_j be the exact tail and $\hat{q}_j$ be the approximate tail computed recursively for each job j at v, satisfying $\hat{q}_j \leq \lfloor R q_j \rfloor$. Suppose there is an integral choice of one option from every family and an ordering of the resulting jobs that finishes by T with exact tails q_j. Then LocalApprox has a correct choice of early guess, a feasible LP, and with high probability outputs a schedule that finishes by H.*

The preceding lemma establishes the local approximation guarantee under the assumed bounds on the recursively computed tails. We now express this guarantee in terms of the completion time stored at a DP state.

Algorithm 1 LocalApprox(F, O)

for $T = 0, 1, \ldots, n-1$ **do**
  $H \leftarrow \lfloor (1+\varepsilon)T \rfloor$
  for all jobs j appearing in F or in an option **do**
    $D_j \leftarrow H - \widehat{q}_j$
    $d_j \leftarrow \max\left\{d \in \mathbb{Z}_{\geq 0} : \left\lfloor \left(1+\frac{\varepsilon}{2}\right) d \right\rfloor \leq D_j\right\}$
  end for
  Enumerate every set A of at most K early families
   and one selected option for every $g \in A$
  for all such guesses **do**
    Fix the guessed option for every family $g \in A$
    Restore all unguessed option families to their original form
    for all families $g \notin A$ **do**
      Delete every option containing a job j with $d_j \leq K$
    end for
    if some remaining family has no option **then**
      continue
    end if
    Solve the prefix LP (1)–(3)
    if the LP is feasible **then**
      for $r = 1, 2, \ldots, \Theta(\log n)$ **do**
        for all remaining families $g \notin A$ **do**
          Choose exactly one option $o \in O_g$ with probability $x_{g,o}$
        end for
        Combine these options with the fixed guessed options
        if $L(t) \leq \lfloor \sigma t \rfloor$ for every relevant threshold t **then**
          Order all selected jobs by nondecreasing d_j
          Store the selected options and the resulting order
          return H
        end if
      end for
    end if
  end for
end for
return Failure

▶ **Corollary 3.12.** *Consider a state S corresponding to a vertex v. Let $B(S) = q_v$ denote the exact minimum time needed to broadcast the entire structure rooted at S, and let $\widetilde{B}(S) = \hat{q}_v$ denote the approximate completion time returned by the algorithm at S.*

If $\hat{q}_j \le \lfloor Rq_j \rfloor$ for every job j at v, then, conditional on these supplied tails, Lemma 3.11 gives $\Pr\left[\widetilde{B}(S) \le \lfloor RB(S) \rfloor\right] \ge 1 - n^{-2}$, where the probability is over the random choices made in the current call to LocalApprox.

3.4.2 PathTail: Computing the Tail Value Along a Path

When we delete an edge from a child cycle C of v, according to Claim 3.6, the resulting option creates at most two jobs. Therefore, every chosen option creates one or two paths starting from v. Consider one such path $P = v = u_0, u_1, \ldots, u_r$. The job associated with u_0 is the call $u_0 \to u_1$. Hence, the tail computed for u_1 must account for all the calls needed to complete the branch along this path up to u_r, including the subtrees attached to its vertices. We compute this tail value backward. Notice that at the last vertex u_r, there is no continuation job along the current cycle path. Therefore, we first call $DP(u_r, C, \perp)$. However, u_r may have option families from other child cycles and fixed jobs from its child bridges. These are already considered in the computation of $DP(u_r, C, \perp)$. We store the returned value as $z_r = DP(u_r, C, \perp)$. Now, consider the previous vertex u_{r-1}. The value z_r represents the tail of the continuation job $u_{r-1} \to u_r$ along the parent cycle C. Therefore, we call $DP(u_{r-1}, C, z_r)$ and store the returned value as $z_{r-1} = DP(u_{r-1}, C, z_r)$. Continuing backward in this way, for each $i = r-1, r-2, \ldots, 1$, we compute $z_i = DP(u_i, C, z_{i+1})$. We define this backward computation as the procedure, $\text{PathTail}(u_1, u_2, \ldots, u_r, C) = z_1$. Hence, the call $u_0 \to u_1$ has tail z_1, which accounts for completing the entire path branch after u_1 becomes informed.

Algorithm 2 PathTail$(u_1, \ldots, u_r, C)$

```
z ← DP(u_r, C, ⊥)
for i = r − 1, r − 2, ..., 1 do
    z ← DP(u_i, C, z)
end for
return z
```

▶ **Lemma 3.13 (♣).** *Consider a directed path arising from a cycle C, $v \to u_1 \to \cdots \to u_r$. Assume that, for every block attached to $u_1, \ldots, u_r$ outside C, the algorithm has already computed a valid schedule whose approximate tail is at most R times its corresponding exact tail. Then $\text{PathTail}(u_1, \ldots, u_r, C)$ represents a valid schedule for the entire path branch. Moreover, if q^*_P denotes the exact tail of this branch, then, with high probability, $\text{PathTail}(u_1, \ldots, u_r, C) \le \lfloor Rq^*_P \rfloor$.*

3.4.3 CycleOptions: Constructing the Option Family for Each Child Cycle

The previously described LocalApprox procedure takes (F, O) as input and returns the completion time of the schedule constructed for the subtree rooted at v, measured from the time v becomes informed. Here, CycleOptions constructs the option family $O_C \in O$ for a child cycle C of v.

The algorithm works as follows. For every edge $e \in E(C)$, we consider the paths obtained by deleting e from C. If the deletion creates two paths P_1 and P_2 starting from v, we compute $q_1 = \text{PathTail}(P_1, C)$ and $q_2 = \text{PathTail}(P_2, C)$. If the deletion creates only one path P starting from v, we compute $q = \text{PathTail}(P, C)$. The purpose of this procedure is to store the corresponding option in O_C, containing two jobs with tails q_1 and q_2 in the first case, or one job with tail q in the second case. Each option corresponds to the choice of an edge e to delete from C.

Algorithm 3 CycleOptions(C, v)

```
O_C ← ∅
for all e ∈ E(C) do
    Delete e conceptually from C
    if the remaining part of C consists of two paths v → P_1 and v → P_2 then
        q_1 ← PathTail(P_1, C)
        q_2 ← PathTail(P_2, C)
        Add the two-job option {q_1, q_2} to O_C
    else
        Let P be the unique remaining path starting at v
        q ← PathTail(P, C)
        Add the one-job option {q} to O_C
    end if
end for
return O_C
```

3.5 Global DP States

Now, we are in a position to construct the DP table by processing the block-incidence tree T_{tree} in a bottom-up manner.

For each original vertex v, we consider a state $DP(v, P, z)$, where P is the parent block of v and z is the continuation tail, if one exists. The state stores the completion time of the schedule constructed at v, measured from the time v becomes informed. Using the previously defined set of fixed jobs F and collection of option families O, we define

$$DP(v, P, z) = \begin{cases} 0, & \text{if } F = \emptyset \text{ and } O = \emptyset, \\ \text{LocalApprox}(F, O), & \text{otherwise.} \end{cases}$$

Each state $DP(v, P, z)$ is computed only once, and its returned value and local schedule are stored. Every later request for the same state uses the stored result. For each original vertex v, we consider a state $DP(v, P, z)$, where P is the parent block of v and z is the continuation tail, if one exists. The state stores the completion time of the schedule constructed at v, measured from the time v becomes informed

Finally, at the source vertex s, there is no parent block or continuation job. Therefore, we compute $DP(s, \perp, \perp)$, which gives the completion time of the constructed broadcast schedule for the entire graph.

Algorithm 4 $\mathrm{DP}(v, P, z)$

```
F ← ∅                                                    ▷ Fixed jobs
O ← ∅                                                    ▷ Option families
for all child bridges B = vu of v with B ≠ P do
    q ← DP(u, B, ⊥)
    Add one fixed job of tail q to F
end for
for all child cycles C of v with C ≠ P do
    O_C ← CycleOptions(C, v)
    Add the family O_C to O
end for
if z ≠ ⊥ then
    Add one fixed continuation job of tail z to F
end if
h ← LocalApprox(F, O)
Store the successful local witness
return h
```

3.6 Correctness

Let $T = b^*(v, P, z)$ denote the exact minimum time needed to complete the structure rooted at v, with z as the exact continuation tail. We show that the global DP returns a value at most $\lfloor Rb^*(v, P, z) \rfloor$.

▶ **Lemma 3.14 (♣).** *For any DP state, if the approximate continuation tail satisfies $\tilde{z} \leq \lfloor Rz \rfloor$, whenever a continuation exists, then, with high probability, $DP(v, P, \tilde{z}) \leq \lfloor Rb^*(v, P, z) \rfloor$.*

The preceding lemma establishes the approximation guarantee for individual DP states. We now apply this guarantee at the root to obtain a valid broadcast schedule for the entire graph.

▶ **Lemma 3.15 (♣).** *Suppose T is such that the given instance G admits a broadcast schedule of length at most T. Then, with high probability, the DP at the root constructs a valid broadcast schedule of length at most $H = \lfloor RT \rfloor$.*

The preceding lemmas establish the correctness and approximation guarantee of the algorithm. It remains to analyze the number of DP states and the running time of the local approximation procedure.

3.7 Running Time

To form the Block incidence tree it takes $\mathcal{O}(n)$ time as total number of vertices and total number of blocks in the cactus can be at most n. Inside *Localapprox*, the algorithm chooses at most K option families. For every chosen family it gueses exactly one option, and a family can contain at most n options. Thus the upper bound on these gueses are $n^K.n^K = n^{\mathcal{O}(K)}$. This is the term which dominates the total runtime.

Now, for one fixed guess, the LP has polynomially many variables and constraints for each distinct relevant deadline t. Also the randomized rounding is repeated $\mathcal{O}(logn)$ times. In each local state, T runs from 0 to $n-1$ at most. So, one call of *Localapprox* takes $n^{\mathcal{O}(1)}.\mathcal{O}(log(n)).n^K = n^{\mathcal{O}(K)}$.

For every vertex v, we eventually need to compute its DP state $DP(v, P, z)$. There are polynomially many relevant states (the continuation tail z is an integer bounded polynomially in n). Thus the total number of DP states is $n^{\mathcal{O}(1)}$. Since, $K = \mathcal{O}(\epsilon^{-2}log(1/\epsilon))$, total runtime we obtain, $n^{\mathcal{O}(\epsilon^{-2}log(1/\epsilon))}$.

This completes the proof of Theorem 1.1.

4 A QPTAS for Telephone Broadcasting on Graphs of Bounded Treewidth

In this section, we study the Telephone Broadcasting problem on graphs of bounded treewidth and present a QPTAS, motivated by the fact that the problem is NP-hard even on graphs of treewidth two [10, 1].

4.1 Algorithm Overview

Egami et al. [5] gave an exact algorithm for Telephone Broadcasting with running time $2^{\mathcal{O}(wt)}n^{\mathcal{O}(1)}$, where t is the target broadcast time and w is the treewidth. Here, we follow a similar dynamic programming approach but aim to avoid the exponential dependence on t. To achieve this, we first partition the available rounds into L consecutive buckets. Each bucket I_k starts at round a_k and ends at U_k, where $U_k = \min\{\lfloor(1+\epsilon)a_k\rfloor, n-1\}$. We then construct a nice edge tree decomposition $\mathcal{T}$ of the graph and apply a bottom-up dynamic programming algorithm. The algorithm considers the possible states at each type of node in $\mathcal{T}$ and combines them to construct a broadcast schedule within the required approximation bound.

The complete algorithm, its correctness analysis, and the running-time analysis are deferred to Appendix B.

5 Conclusion

In this paper, we studied the Telephone Broadcasting problem on cactus graphs and graphs of bounded treewidth. We presented a randomized PTAS for cactus graphs and a deterministic QPTAS for graphs of bounded treewidth. Our algorithms exploit the underlying graph structure to obtain broadcast schedules arbitrarily close to optimal, despite the NP-hardness of the problem on these graph classes. In particular, our result for cactus graphs addresses an open question posed by Aminian et al. in the randomized setting. An interesting direction for future work is to obtain a deterministic PTAS for cactus graphs and to investigate whether the quasi-polynomial running time for bounded-treewidth graphs can be improved to polynomial time.

6 AI-Assisted Preparation

OpenAI was used as interactive research and writing tools during the development of this work. The initial algorithmic idea arose through an interactive discussion with ChatGPT 5.6 sol; the authors subsequently developed and reformulated the complete approach and independently verified all algorithms, proofs, and mathematical claims. This tool was later used to formalize author-provided identify typographical errors and minor logical inconsistencies. All AI-assisted content was critically reviewed and revised by the authors, who take sole responsibility for the results, citations, and final text.

References

1. Aida Aminian, Shahin Kamali, Seyed-Mohammad Seyed-Javadi, and Sumedha. On the Complexity of Telephone Broadcasting from Cacti to Bounded Pathwidth Graphs. In Keren Censor-Hillel, Fabrizio Grandoni, Joël Ouaknine, and Gabriele Puppis, editors, *52nd International Colloquium on Automata, Languages, and Programming (ICALP 2025)*, volume 334 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 10:1–10:17, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: `https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICALP.2025.10`, `doi:10.4230/LIPIcs.ICALP.2025.10`.
2. Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. *Concentration Inequalities - A Nonasymptotic Theory of Independence*. Oxford University Press, 2013. URL: `https://doi.org/10.1093/acprof:oso/9780199535255.001.0001`, `doi:10.1093/ACPROF:OSO/9780199535255.001.0001`.
3. Jeffrey Bringolf, Hovhannes A Harutyunyan, Shahin Kamali, and Seyed-Mohammad Seyed-Javadi. On hardness and approximation of broadcasting in sparse graphs. *arXiv e-prints*, pages arXiv–2510, 2025.
4. Maja Čevnik and Janez Žerovnik. Broadcasting on cactus graphs. *Journal of Combinatorial optimization*, 33(1):292–316, 2017.
5. Yudai Egami, Tatsuya Gima, Tesshu Hanaka, Yasuaki Kobayashi, Michael Lampis, Valia Mitsou, Edouard Nemery, Yota Otachi, Manolis Vasilakis, and Daniel Vaz. Broadcasting Under Structural Restrictions. In Paweł Gawrychowski, Filip Mazowiecki, and Michał Skrzypczak, editors, *50th International Symposium on Mathematical Foundations of Computer Science (MFCS 2025)*, volume 345 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 42:1–42:18, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: `https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.MFCS.2025.42`, `doi:10.4230/LIPIcs.MFCS.2025.42`.
6. Michael Elkin and Guy Kortsarz. Combinatorial logarithmic approximation algorithm for directed telephone broadcast problem. In *Proceedings of the thiry-fourth annual ACM symposium on Theory of computing*, pages 438–447, 2002.
7. Michael Elkin and Guy Kortsarz. Sublogarithmic approximation for telephone multicast. *Journal of Computer and System Sciences*, 72(4):648–659, 2006.
8. Fedor V Fomin, Pierre Fraigniaud, and Petr A Golovach. Parameterized complexity of broadcasting in graphs. *Theoretical Computer Science*, 997:114508, 2024.
9. Peter J. Slater, Ernest J. Cockayne, and Stephen T. Hedetniemi. Information dissemination in trees. *SIAM Journal on Computing*, 10(4):692–701, 1981.
10. Prafullkumar Tale. Telephone broadcast on graphs of treewidth two. *Theoretical Computer Science*, 1045:115282, 2025.

A Deferred Proofs for the the Cactus PTAS

▶ **Observation 3.2 (♣[2]).** *Let $G(V, E)$ be a connected cactus. A spanning tree of G is obtained by deleting exactly one edge from every cycle and keeping every bridge. Conversely, deleting one edge from every cycle produces a spanning tree.*

Proof. Let $\mathcal{C}$ be the set of cycles of G. For each $C \in \mathcal{C}$, choose an edge $e_C \in E(C)$, and let $G' = G - \{e_C : C \in \mathcal{C}\}$. By the definition of a cactus, every edge belongs to at most one cycle.

We first show that G' is connected. Let $e_C = xy$ be the edge deleted from a cycle C. The graph $C - e_C$ contains an $x - y$ path. Moreover, no edge of this path is deleted for another cycle, since an edge cannot belong to two different cycles. Thus, the endpoints of every deleted edge remain connected in G'. Since G is connected, G' is connected.

We next show that G' is acyclic. Suppose, for contradiction, that G' contains a cycle D. Then D is also a cycle of G. However, the chosen edge $e_D \in E(D)$ was deleted when constructing G', contradicting $D \subseteq G'$. Therefore, G' is acyclic. Hence G' is connected and acyclic, and therefore it is a spanning tree of G.

Conversely, let T be a spanning tree of G. The tree T must omit at least one edge from every cycle; otherwise, T would contain a cycle. Moreover, T cannot omit a bridge, since deleting a bridge disconnects G.

Let $k = |\mathcal{C}|$. By the first part, deleting one edge from each of the k cycles produces a spanning tree. Consequently, $|E(G)| - k = |V(G)| - 1$. Every spanning tree has exactly $|V(G)| - 1$ edges and therefore omits exactly k edges of G. Since T must omit at least one edge from each of the k cycles and cannot omit any bridge, it omits exactly one edge from every cycle. ◀

▷ Claim 3.6 (♣). Every option $o_e \in O_C$, corresponding to the deletion of an edge e from a child cycle C of v, creates at most two jobs at v.

Proof. Since C is a simple cycle, v has exactly two neighbors in C. After deleting e, the graph $C - e$ is a simple path containing v.

If e is incident to v, then v is an endpoint of $C - e$. Thus, only one path starts from v, and v makes exactly one call into C. Hence, the corresponding option contains one job.

If e is not incident to v, then v is an internal vertex of $C - e$. In this case, the path extends from v in two directions. Therefore, v makes one call in each direction, and the corresponding option contains two jobs. Hence, every option $o_e \in \mathcal{O}_C$ contains at most two jobs. ◀

▶ **Observation 3.7 (♣).** *In any feasible integral schedule, every job j with deadline d_j, at most K elements of $\mathcal{O}$ can have their selected option (edge) that creates a job with deadline (d_j) at most K.*

Proof. Call a family early if its selected option contains a job j with $d_j \leq K$. Every early family therefore contributes at least one distinct selected job that must be scheduled within the first K positions. Since all jobs have unit processing time, a feasible schedule has only K available positions before time K. Hence it can contain at most K such jobs, and consequently at most K early families. ◀

[2] ♣ Proofs of statements marked with this symbol are deferred to the appendix.

▶ **Lemma 3.9 (♣).** *For a sufficiently large constant C in the definition of K, one application of randomized rounding to a feasible LP solution satisfies $L(t) \le \lfloor \sigma t \rfloor$ simultaneously for all relevant thresholds t, with non-zero probability.*

Proof. For every relevant threshold $t \le K$, only fixed jobs contribute to $L(t)$. Hence, the LP constraints give $L(t) = F(t) \le t \le \lfloor \sigma t \rfloor$ deterministically.

Now fix a relevant threshold $t > K$. The random variables $Y_C(t)$, for $C \notin A$, are independent across option families and take values in $\{0, 1, 2\}$. By (4), $\mathbb{E}[L(t)] \le t$. Moreover,

$$\sum_{C \notin A} \mathrm{Var}(Y_C(t)) \le 2 \sum_{C \notin A} \mathbb{E}[Y_C(t)] \le 2t.$$

Since $L(t)$ is an integer and $\sigma = 1 + \epsilon/2$, the event $L(t) > \lfloor \sigma t \rfloor$ implies $L(t) - \mathbb{E}[L(t)] > \frac{\epsilon t}{2}$. Applying Bernstein's inequality [2] to the centered variables $Y_C(t) - \mathbb{E}[Y_C(t)]$, we obtain $\Pr[L(t) > \lfloor \sigma t \rfloor] \le \exp(-c\epsilon^2 t)$ for an absolute constant $c > 0$.

Let B_t denote the event $L(t) > \lfloor \sigma t \rfloor$, and let $\mathcal{D}$ be the set of relevant thresholds. By the union bound,

$$\Pr\left[\bigcup_{\substack{t \in \mathcal{D} \\ t > K}} B_t\right] \le \sum_{\substack{t \in \mathcal{D} \\ t > K}} e^{-c\epsilon^2 t} \le \sum_{t=K+1}^{\infty} e^{-c\epsilon^2 t} = \frac{e^{-c\epsilon^2(K+1)}}{1 - e^{-c\epsilon^2}}.$$

For $K = \lceil C\epsilon^{-2} \log(1/\epsilon) \rceil$ and sufficiently large C, this quantity is at most $1/4$. Therefore, one rounding trial satisfies $L(t) \le \lfloor \sigma t \rfloor$ simultaneously for all relevant thresholds with probability at least $3/4$.

To increase the success probability, we repeat the rounding independently $r = \lceil \log_2 n \rceil$ times, checking all relevant thresholds after each trial. The probability that every trial fails is at most $(1/4)^r \le n^{-2}$. Thus, with probability at least $1 - n^{-2}$, at least one trial satisfies all the required inequalities. ◀

▷ Claim 3.10 (♣). Suppose that the selected jobs are ordered by nondecreasing transformed deadlines d_j, and $L(t) \le \lfloor \sigma t \rfloor$ for every relevant threshold t. Then every selected job j satisfies $x_j \le D_j$. Consequently, $x_j + \hat{q}_j \le H$.

Proof. Consider the selected jobs ordered by nondecreasing transformed deadlines d_j. Let x_j be the position of job j in this ordering. Since every job appearing before j has transformed deadline at most d_j, and $L(t)$ counts the number of jobs with transformed deadlines at most t, we have $x_j \le L(d_j)$.

By Lemma 3.9 and the definition of d_j, $x_j \le L(d_j) \le \lfloor \sigma d_j \rfloor \le D_j$. Finally, since $D_j = H - \hat{q}_j$, we obtain $x_j + \hat{q}_j \le D_j + \hat{q}_j = H$. Hence, every selected job satisfies the required completion-time bound. ◀

▶ **Lemma 3.11 (♣).** *Consider a local instance at some vertex v. Let q_j be the exact tail and $\hat{q}_j$ be the approximate tail computed recursively for each job j at v, satisfying $\hat{q}_j \le \lfloor Rq_j \rfloor$. Suppose there is an integral choice of one option from every family and an ordering of the resulting jobs that finishes by T with exact tails q_j. Then LocalApprox has a correct choice of early guess, a feasible LP, and with high probability outputs a schedule that finishes by H.*

Proof. Consider an exact integral solution, and let x_j be the position of job j in that solution. Then clearly,$x_j + q_j \le T$. By the definition of D_j, $D_j = H - \hat{q}_j \ge \lfloor R(T - q_j) \rfloor \ge \lfloor Rx_j \rfloor \ge \lfloor \sigma x_j \rfloor$. Again, we have $d_j = \max\{d \in \mathbb{Z}_{\ge 0} : \lfloor \sigma d \rfloor \le D_j\}$. This implies $x_j \le d_j$.

Now, we can always guess the correct choice of early families A of size at most K by enumeration. We now explicitly construct a feasible solution to the prefix LP. For every remaining family $C \notin A$, set

$$x_{C,e} = \begin{cases} 1, & \text{if } J_e \text{ is the chosen option,} \\ 0, & \text{otherwise.} \end{cases}$$

For any threshold t, all jobs with $d_j \le t$ occur in the exact ordering within the first t positions, since $x_j \le d_j \le t$. Hence, there are at most t such jobs. Therefore, the integral solution satisfies all LP constraints, and the LP is feasible.

By Lemma 3.9, the repeated rounding for this feasible LP succeeds with probability at least $1 - n^{-2}$. If LocalApprox returns before this guess is reached, its returned completion-time bound is already at most H. Otherwise, successful rounding and the ordering described in Claim 3.10 give a valid local schedule in which every branch finishes within H rounds.

◀

▶ **Lemma 3.13 (♣).** *Consider a directed path arising from a cycle C, $v \to u_1 \to \cdots \to u_r$. Assume that, for every block attached to $u_1, \ldots, u_r$ outside C, the algorithm has already computed a valid schedule whose approximate tail is at most R times its corresponding exact tail. Then* $\mathrm{PathTail}(u_1, \ldots, u_r, C)$ *represents a valid schedule for the entire path branch. Moreover, if q_P^* denotes the exact tail of this branch, then, with high probability,* $\mathrm{PathTail}(u_1, \ldots, u_r, C) \le \lfloor R q_P^* \rfloor$.

Proof. For each i, let z_i denote the exact minimum time needed to complete the branch rooted at u_i after u_i becomes informed. Let $b^*(v, P, z)$ denote the exact minimum completion time at v when the exact continuation tail is z.

At the endpoint u_r, there is no continuation through C. Therefore, $z_r = b^*(u_r, C, \perp)$. = For every $i = 1, 2, \ldots, r-1$, the call $u_i \to u_{i+1}$ is compulsory and has exact continuation tail z_{i+1}. Hence, $z_i = b^*(u_i, C, z_{i+1})$. Finally, the exact tail of the complete path branch is $q_P^* = z_1$.

We prove by backward induction that, for every $i = 1, 2, \ldots, r$, $\tilde{z}_i \le \lfloor R z_i \rfloor$. At the endpoint u_r, there is no continuation through C. All jobs in the local instance at u_r arise from blocks attached outside C. By the induction hypothesis on these attached blocks, each such job j has a valid approximate tail satisfying $\hat{q}_j \le \lfloor R q_j \rfloor$. Therefore, by Lemma 3.11, we obtain $\tilde{z}_r = DP(u_r, C, \perp) \le \lfloor R z_r \rfloor$.

Now, suppose the claim holds at u_{i+1}. At u_i, the compulsory call $u_i \to u_{i+1}$ is represented by a continuation job. Its exact tail is z_{i+1}, and its approximate tail is $\tilde{z}_{i+1}$. By the induction hypothesis, $\tilde{z}_{i+1} \le \lfloor R z_{i+1} \rfloor$. Thus, this continuation job satisfies the required factor-R bound.

Every other job at u_i arises from a block attached outside C, and its approximate tail satisfies the same bound by the induction hypothesis on these attached blocks. Consequently, every job supplied to the local instance at u_i satisfies the hypothesis of Lemma 3.11. Hence, $\tilde{z}_i = DP(u_i, C, \tilde{z}_{i+1}) \le \lfloor R z_i \rfloor$. Therefore, the claim holds for every $i = 1, 2, \ldots, r$. Consequently,

$$\mathrm{PathTail}(u_1, u_2, \ldots, u_r, C) = \tilde{z}_1 \le \lfloor R z_1 \rfloor = \lfloor R q_P^* \rfloor.$$

◀

▶ **Lemma 3.14 (♣).** *For any DP state, if the approximate continuation tail satisfies $\tilde{z} \le \lfloor Rz \rfloor$, whenever a continuation exists, then, with high probability, $DP(v, P, \tilde{z}) \le \lfloor R b^*(v, P, z) \rfloor$.*

Proof. Consider the local instance constructed by $DP(v, P, \tilde{z})$.

First, consider a child bridge of v. Its tail has already been computed, since the algorithm processes the block-incidence tree in a bottom-up manner. Hence, by the induction hypothesis, its approximate tail is at most R times its corresponding exact tail.

Next, consider a child cycle of v. For every possible deleted edge of this cycle, the algorithm constructs the corresponding path or paths and computes their tails using PathTail. By Lemma 3.13, every such approximate path tail is at most R times its corresponding exact tail. Finally, if the current state contains a continuation tail, then, by the assumption of the lemma, $\tilde{z} \leq \lfloor Rz \rfloor$. Consequently, every tail supplied to the current local instance satisfies $\hat{q}_j \leq \lfloor Rq_j \rfloor$.

Let $T = b^*(v, P, z)$. By the definition of T, there exists an exact local solution that selects one option from every child cycle family and orders the resulting jobs so that every branch finishes within T rounds.

Since LocalApprox considers every possible target value, it considers this value of T and sets $H = \lfloor RT \rfloor$. All tails supplied to this local instance satisfy the hypothesis of Lemma 3.11. Therefore, by Lemma 3.11, with high probability, the randomized rounding produces a valid local schedule in which every branch finishes within H rounds. Hence, $DP(v, P, \tilde{z}) \leq H = \lfloor RT \rfloor$. Substituting $T = b^*(v, P, z)$, we obtain $DP(v, P, \tilde{z}) \leq \lfloor Rb^*(v, P, z) \rfloor$. ◀

▶ **Lemma 3.15 (♣).** *Suppose T is such that the given instance G admits a broadcast schedule of length at most T. Then, with high probability, the DP at the root constructs a valid broadcast schedule of length at most $H = \lfloor RT \rfloor$.*

Proof. Since T is a yes-value, there exists a broadcast schedule of length at most T. Consider the broadcast tree induced by this schedule. Exactly one edge from every cycle is omitted from this tree. The randomized rounding does not necessarily select the same combination of deleted edges. However, by Lemma 3.11, with high probability, it produces a feasible local schedule within the required completion-time bound.

Now, consider the bottom-up DP computation from the leaves toward the root. For every rooted substructure, the algorithm considers the corresponding local instance. By Lemma 3.14, provided the required local computations succeed, the value computed for each corresponding state is at most R times its exact value.

Consequently, at the root, we obtain, $\mathrm{DP}(s, \perp, \perp) \leq \lfloor Rb(G, s) \rfloor \leq \lfloor RT \rfloor = H$. Since there is no parent block or continuation at the source, $\mathrm{DP}(s, \perp, \perp)$ represents the completion-time bound of the constructed broadcast schedule for the entire graph.

Therefore, when the required local computations succeed, the algorithm constructs a valid broadcast schedule of length at most H. ◀

B Algorithmic Details and Proofs for the Treewidth QPTAS

B.1 Algorithm

We now describe the steps of our algorithm, starting with the choice of the target broadcast time and the partitioning of rounds into consecutive buckets.

Step 1. We first consider each target value $t \in \{0, 1, \ldots, n-1\}$ and set $H = \lfloor (1+\epsilon)t \rfloor$. For a fixed choice of t, our aim is to construct a broadcast schedule that finishes within H rounds. If no such schedule is found, we move to the next value of t.

Step 2. We partition the available rounds into L consecutive groups, denoted by $I_1, I_2, \ldots, I_L$. Each group I_k starts at round a_k and ends at U_k, where $I_k = \{a_k, a_k+1, \ldots, U_k\}$, where $U_k =$

$\lfloor(1+\epsilon)a_k\rfloor$. The next group starts at $a_{k+1} = U_k+1$. Thus, $I_{k+1} = \{U_k+1, U_k+2, \ldots, U_{k+1}\}$, where $U_{k+1} = \lfloor(1+\epsilon)(U_k+1)\rfloor$. We start with $a_1 = 1$ and continue this process until all rounds up to t are covered, adjusting the endpoint of the last group to t if necessary.

▶ **Observation B.1.** *The value of L can be atmost $\mathcal{O}(logn/\epsilon)$, as t can be atmost $n-1$.*

Proof. By construction, the first bucket starts at $a_1 = 1$. For every $k < L$, we have $a_{k+1} = U_k + 1 = \lfloor(1+\epsilon)a_k\rfloor + 1 > (1+\epsilon)a_k$. Applying this inequality repeatedly, we obtain $a_k > (1+\epsilon)^{k-1}$ for every $k \geq 2$. Since the last bucket starts at $a_L \leq t \leq n-1$, it follows that, for $L \geq 2$, $(1+\epsilon)^{L-1} < a_L \leq t \leq n-1$. Taking logarithms, we get $L - 1 < \frac{\log n}{\log(1+\epsilon)}$. For $0 < \epsilon \leq 1$, we have $\log(1+\epsilon) \geq \epsilon/2$. Therefore, $L < 1 + \frac{2\log n}{\epsilon} = \mathcal{O}\left(\frac{\log n}{\epsilon}\right)$. The bound is immediate when $L \leq 1$. ◀

The preceding observation bounds the number of buckets by $(\log n/\epsilon)$. We now show that only a bounded number of these buckets can contain a single call rank.

▶ **Observation B.2.** *There are at most $\mathcal{O}(1/\epsilon)$ singleton buckets.*

Proof. A bucket starting at a is a singleton exactly when $U = \lfloor(1+\epsilon)a\rfloor = a$. Since $(1+\epsilon)a = a + \epsilon a$, this holds if and only if $\epsilon a < 1$, or equivalently, $a < \frac{1}{\epsilon}$.

Therefore, only buckets with starting values $a < 1/\epsilon$ can be singletons. Since the starting values are distinct positive integers, there are at most $\lceil 1/\epsilon\rceil - 1$ such buckets. Hence, the number of singleton buckets is $\mathcal{O}(1/\epsilon)$. 1 ◀

Step3. We assume that a nice edge tree decomposition $\mathcal{T}$ of G is given. This decomposition consists of five types of nodes: leaf, introduce vertex, introduce edge, forget vertex, and join nodes. We consider the possible states at each type of node and construct a DP table containing all valid states.

Consider a node $x \in V(\mathcal{T})$, and let B_x denote its bag. For each vertex $v \in B_x$, we maintain the tuple $\big(T(v), p(v), d_1(v), d_2(v), \ldots, d_L(v)\big)$. Here, $T(v) \in \{0, 1, \ldots, H\}$ is an upper bound on the round in which v becomes informed in the reconstructed broadcast schedule. We set $T(s) = 0$ and require $T(v) \in \{1, \ldots, H\}$ for every non-source vertex. Also $p(v) \in \{0, 1\}$ indicates whether a parent edge of v has been selected among the edges introduced so far. For every $k \in [L]$, the counter $d_k(v)$ records the number of selected outgoing calls from v assigned to bucket I_k. These buckets contain local call ranks, measured from the time at which the new vertex becomes informed. The DP table D_x stores the feasible combinations of these values for all vertices in B_x.

▶ **Observation B.3.** *For every vertex $v \in B_x$ and every $k \in [L]$, we have $0 \leq d_k(v) \leq c_k$, where $c_k = |I_k|$.*

Next, we consider the different types of nodes in the tree decomposition. For each node x, we construct the DP table $\mathcal{D}_x$ containing all feasible states.

Leaf node. For an empty leaf node x, the DP table $\mathcal{D}_x$ contains only the empty state.

Introduce vertex node. Suppose a new vertex v is introduced at node x. For each state in the child table, we consider every possible activation time bound $T(v) \in \{1, 2, \ldots, H\}$. Since no edge incident with v has been introduced at this node, we initialize $p(v) = 0$, $d_k(v) = 0$ for every $k \in [L]$. The source $v = s$, has the fixed activation-time bound, $T(s) = 0$. In this case, we set

$$p(s) = 0, \quad d_k(s) = 0 \quad \text{for every } k \in [L].$$

Introduce edge node. Suppose the edge uv is introduced at node x. For every state in the child table, we consider the following three cases.

Case 1. The edge uv is not used to inform either endpoint. In this case, the state remains unchanged and is carried forward to $\mathcal{D}_x$.

Case 2. Suppose u informs v through the edge uv. For a guessed bucket $k \in [L]$, this case is considered only if $T(u) < T(v)$, $d_k(u) < c_k$, $p(v) = 0$, and $T(v) = T(u) + U_k$.

If all these conditions are satisfied, we create a new state by updating $p(v) \leftarrow 1$, and $d_k(u) \leftarrow d_k(u) + 1$. All other values remain unchanged. Otherwise, we discard this case.

Case 3. Suppose v informs u through the edge uv. Similarly, for a guessed bucket $k \in [L]$, this case is considered only if $T(v) < T(u)$, $d_k(v) < c_k$, $p(u) = 0$, and $T(u) = T(v) + U_k$.

If all these conditions are satisfied, we create a new state by updating $p(u) \leftarrow 1$, and $d_k(v) \leftarrow d_k(v) + 1$. All other values remain unchanged. Otherwise, we discard this case.

Forget vertex node. Suppose a vertex v is forgotten at node x. By the properties of a nice tree decomposition, v does not appear in any bag above x, and all edges incident with v have already been introduced. Therefore, no further call involving v can be assigned.

For every non-source vertex $v \neq s$, we retain only those states for which $p(v) = 1$, ensuring that v has already been assigned a parent in the broadcast tree. For the source vertex s, we require $p(s) = 0$. In each retained state, we remove all information associated with v.

▶ **Observation B.4.** *forgetoperation The above forget operation is safe.*

Proof. Once a vertex v is forgotten, it does not appear in any ancestor bag. Hence, every edge incident with v has already been considered, and no new call involving v can be introduced later.

Moreover, the DP retains a non-source vertex only when $p(v) = 1$, ensuring that a parent has already been assigned to v. For the source, we maintain $p(s) = 0$. Therefore, all information required for processing v has been considered, and its state information can be removed. ◀

Join node: Suppose x is a join node with two children x_1 and x_2, where $B_x = B_{x_1} = B_{x_2}$. We consider a pair of states, one from $\mathcal{D}_{x_1}$ and the other from $\mathcal{D}_{x_2}$. These states can be combined only if, for every vertex $v \in B_x$, we have $T_{x_1}(v) = T_{x_2}(v)$, $p_{x_1}(v) + p_{x_2}(v) \leq 1$, and $d_k^x(v) = d_k^{x_1}(v) + d_k^{x_2}(v) \leq c_k$ for every $k \in [L]$.

If these conditions are satisfied, we create a new state in $\mathcal{D}_x$ with $T_x(v) = T_{x_1}(v) = T_{x_2}(v)$ and $p_x(v) = p_{x_1}(v) + p_{x_2}(v)$. The bucket counts are updated as given above.

Finally, if the DP table at the root contains a feasible state in which every non-source vertex has been assigned a parent and all the required constraints are satisfied, the algorithm accepts the corresponding target broadcast time.

▶ **Lemma B.5.** *Every valid DP computation at the root induces a spanning tree of G rooted at the source s.*

Proof. Suppose there is a valid DP state at the root of the nice edge tree decomposition. By the DP transitions, once $p(v)$ becomes 1, no further parent can be assigned to v. Moreover, at a forget node, every non-source vertex v is required to satisfy $p(v) = 1$. Hence, every vertex $v \neq s$ has exactly one parent, while the source s has no parent. Consider the directed graph formed by all selected edges at the introduce-edge nodes, oriented from the informing vertex to the informed vertex. We show that this graph is a spanning tree rooted at s.

Suppose, for contradiction, it contains a directed cycle $u_0 \to u_1 \to \cdots \to u_r \to u_0$. By the introduce-edge transitions, the labels $T(v)$ strictly increase along every selected directed edge. Therefore, $T(u_0) < T(u_1) < \cdots < T(u_r) < T(u_0)$, which is impossible. Hence, the selected directed graph contains no directed cycle. Now, consider any vertex $v \neq s$ and repeatedly follow its parent. Since every non-source vertex has exactly one parent and there are no directed cycles, this sequence must eventually reach a vertex with no parent. As s is the only such vertex, the sequence terminates at s. Thus, every vertex is connected to s through the selected edges.

Consequently, the selected graph is connected and acyclic, and hence forms a spanning tree of G rooted at s. ◀

The preceding lemma establishes that a valid DP computation induces a spanning tree rooted at s. We next show that the corresponding DP witness also yields a feasible broadcast schedule within the prescribed time bound.

▶ **Lemma B.6.** *Every accepted DP witness can be converted into a valid broadcast schedule that finishes within H rounds.*

Proof. By Lemma B.5, the selected edges form a spanning tree rooted at the source s. For a vertex u and a bucket I_k, let $C_k(u)$ denote the children of u whose selected edges are assigned to bucket I_k. The introduce-edge and join transitions maintain the bucket capacities. Since all incident edges have been processed before a vertex is forgotten, these constraints ensure that $|C_k(u)| \leq c_k = |I_k|$ for every vertex u and every $k \in [L]$.

We can therefore assign each child $v \in C_k(u)$ a distinct local call rank $r_{uv} \in I_k$. Since the buckets are pairwise disjoint, all call ranks assigned to the children of a fixed vertex u are distinct. Define the actual activation times recursively along the selected tree by $a(s) = 0$, $a(v) = a(u) + r_{uv}$ for every selected edge $u \to v$. Schedule the call $u \to v$ in round $a(u) + r_{uv}$. Every call rank is positive, so the sender is informed before the round in which it makes the call. Distinct local ranks ensure that each vertex makes at most one call per round. Moreover, every non-source vertex receives the message through its unique parent edge. Hence, this construction gives a valid broadcast schedule.

We prove by induction along the rooted tree that $a(v) \leq T(v)$ for every vertex v. At the source, $a(s) = T(s) = 0$. Suppose that $u \to v$ is assigned to bucket I_k and that $a(u) \leq T(u)$. Since $r_{uv} \leq U_k$ and the introduce-edge transition requires $T(v) = T(u) + U_k$, we obtain $a(v) = a(u) + r_{uv} \leq T(u) + U_k = T(v)$. Thus, the induction holds for every vertex. Finally, every activation-time bound satisfies $T(v) \leq H$. Therefore, $\max_{v \in V(G)} a(v) \leq \max_{v \in V(G)} T(v) \leq H$. The reconstructed broadcast schedule finishes within H rounds.

◀

Together, the preceding two lemmas establish the correctness of the schedule reconstructed from an accepted DP computation. It remains to show that an optimal broadcast tree admits a valid DP computation within the approximate time bound.

▶ **Theorem B.7.** *If there exists an optimal broadcast tree with completion time T, then there exists a valid DP computation that constructs a broadcast schedule of length at most $H = \lfloor (1+\epsilon)T \rfloor$.*

Proof. Suppose there exists an optimal broadcast tree B with completion time T. Consider any vertex v, and let $P = s = v_0, v_1, \ldots, v_q = v$ be the unique path from s to v in B. For each $i \in [q]$, let r_i denote the position of the call $v_{i-1} \to v_i$ in the local ordering of calls made by v_{i-1}. Thus, the actual time at which v becomes informed is $a(v) = \sum_{i=1}^{q} r_i \leq T$.

Our algorithm considers every possible activation-time bound from 0 to H and examines all possible choices at each introduce-edge node. Therefore, it considers the parent-child relationships corresponding to the optimal broadcast tree B. For each edge $v_{i-1}v_i$ of P, let $k_i \in [L]$ be the bucket containing the call rank r_i. By the construction of the buckets, $r_i \le U_{k_i} \le (1+\epsilon)r_i$. In the corresponding DP computation, the activation-time bounds satisfy $T(v_i) = T(v_{i-1}) + U_{k_i}$. Since $T(s) = 0$, we obtain

$$T(v) = \sum_{i=1}^{q} U_{k_i} \le (1+\epsilon)\sum_{i=1}^{q} r_i = (1+\epsilon)a(v) \le (1+\epsilon)T.$$

As $T(v)$ is an integer, $T(v) \le \lfloor (1+\epsilon)T \rfloor = H$. Now, consider any vertex u in the optimal broadcast tree. Since u informs at most one child in each round, all its outgoing calls have distinct ranks. Therefore, for every bucket I_k, the number of children of u assigned to I_k is at most $|I_k| = c_k$. Hence, $d_k(u) \le c_k$ for every $k \in [L]$. Moreover, every non-source vertex has exactly one parent in B, while the source has no parent. Thus, the parent assignments satisfy the introduce-edge, forget, and join conditions of the DP. The activation-time bounds are consistent across the bags, and the bucket counts satisfy the required capacity constraints.

Consequently, the DP considers a valid witness corresponding to B, with every activation-time bound at most H. By Lemma B.5 and B.6, this witness can be converted into a valid broadcast schedule that finishes within H rounds.

Therefore, the constructed broadcast schedule finishes within H rounds. ◀

The preceding lemmas establish the validity of the reconstructed broadcast schedule, while Theorem B.7 guarantees the existence of a DP computation achieving the desired approximation bound. It remains to analyze the number of DP states and the time required to compute them.

B.2 Running Time

Consider a vertex $v \in B_x$. For each such vertex, the DP state stores the information $(T(v), p(v), d_1(v), d_2(v), \ldots, d_L(v))$. Since $T(v) \in \{0, 1, \ldots, H\}$ and $H = \mathcal{O}(n)$, there are $\mathcal{O}(n)$ possible values for $T(v)$. The variable $p(v) \in \{0, 1\}$ has two possible values. Moreover, for every $k \in [L]$, we have $0 \le d_k(v) \le c_k$, where $c_k = |I_k|$. Thus, each $d_k(v)$ has at most $c_k + 1$ possible values. Therefore, the total number of possible configurations for a single vertex v is at most $\mathcal{O}(n) \cdot 2 \cdot \prod_{k=1}^{L}(c_k + 1)$.

Since $c_k \le n$ for every $k \in [L]$, we have $\prod_{k=1}^{L}(c_k + 1) \le (n+1)^L$. Moreover, by Observation B.1, $L = \mathcal{O}\left(\frac{\log n}{\epsilon}\right)$. Hence, the number of possible configurations for one bag vertex is at most $\mathcal{O}(n) \cdot 2 \cdot (n+1)^L = n^{\mathcal{O}\left(\frac{\log n}{\epsilon}\right)}$.

A bag contains at most $w+1$ vertices, since the width of the tree decomposition is w. Therefore, the total number of possible DP configurations at any bag is at most

$$\left(n^{\mathcal{O}\left(\frac{\log n}{\epsilon}\right)}\right)^{w+1} = n^{\mathcal{O}\left(\frac{(w+1)\log n}{\epsilon}\right)}.$$

At a join node, we examine pairs of configurations from the two child tables. Squaring the number of configurations preserves the bound $n^{\mathcal{O}((w+1)\log n/\varepsilon)}$, and each pair is checked in polynomial time. The other transitions require only polynomially many extensions per configuration. Using a nice edge tree decomposition with polynomially many nodes and considering all target values therefore gives the stated total running time.

This completes the proof of Theorem 1.2.